

Using the Type 33 Symbol Generator

This document describes how to use the Type 33 Symbol Senerator in your application. This is briefly described in the DEC PDP-1 Handbook.

What is the Type 33 Symbol Generator

The original was an option for the Type 30 and other displays. It provided a hardware implementation for drawing any symbol that could be represented in a 5x7 dot matrix or a tiling of same. It was an implementation of the same algorithm used in several software drawing programs, but was far faster.

This is implemented as a set of dyamic IOTs, IOT_26 and IOT_27 and requires no changes to the emulator as long as it has the Dynamic IOT extension. Just copy the IOT26-27.c files to the IOTs directory, type 'make', that's it, not even a restart is needed.

These IOTs duplicate the original behavior, including the timing.

How does it work?

Characters are drawin in two parts, a left part and a right part. For each a word whose bits make up part of a 5x7 dot matrix to display is used, therefore each character requires two words.

After the initial positioning successive characters can be drawn. After each character, the y axis is restored to its initial position and the x axis incremented to be the next location to draw in.

Important note

Displayed characters do not remain on the screen, all characters have to be completely redrawn fast enough to be seen.

It is also imperative that the gpl and gpr instructions are properly waited for. Using the i modifier, completion needed, and ioh are essential unless you can be sure that enough time has elapsed between the drawing instructions, e.g. *gpl i; ioh*, or the characters will not draw correctly.

Unfortunately, the timing is variable, averaging about 140 microseconds per character, but it depends upon how many dots have to be drawn, see below.

Usage

There are 6 IOT commands:

-IOT 2007, sdb, position for writing *This is a special operation, see below*

-IOT 2026, glf, load format

-IOT 26, gsp, space one character position right

-IOT 2027, gpl, draw left half of character

-IOT 27, gpr, draw right half of character

-IOT 127, gcf, clear light pen flag, does nothing until light pen support is provided

The sdb instruction

This sets the x and y coordinates of the lower left corner to draw a character from next. Normally, this is a variation of the dpy IOT, IOT 07.

However, this conflicts with one of the bits used to change the display origin. Fortunately, it can be simulated by using *dpy-i 400*, which uses an invisible brightness setting so no unwanted dot will be visible. The only difference between this workaround and the original one is that this takes 35 microseconds to complete, the original takes only 30 microseconds.

If the pidp1 emulator was compiled with Type 33 support, then the authentic instruction is available, but origin shifting will not be.

The IO and AC registers are used.

722b07

b - brightness 0-4, 0 normal, 3 brightest, 4 invisible

IO - the high 10 bits are the 1's complement signed y coordinate, with 0 being the center of the screen

AC - the high 10 bits are the 1's complement signed x coordinate, with 0 being the center of the screen

Completion pulses will ****not**** be sent, don't do a blocking wait.

The glf instruction

This sets the character size and selects automatic spacing between letters.

The IO register is used.

722026

IO - bits 16-17 select the character size, 0-3, bit 15 enables automatic character spacing

Completion pulses will ****not**** be sent, don't do a blocking wait.

The gsp instruction

This positions the next point one character size plus, if auto spacing is on, one character auto spacing width to the right of the last character displayed.

It has no other attributes and does not use the AC or IO registers.

720026

Completion pulses are supported.

The gpl instruction

This draws the left half of a character starting from the current display location.

The IO register is used.

722027

IO - bits 0-16 contain the first 17 bits of the 35 bit character, bit 17 if set makes this character a subscript by lowering its position.

Completion pulses will are supported and should be used.

If subscripting is used, the y position remains set to the proper location for the next half character.

This instruction takes a variable length of time to complete, 2 microseconds for each 0 bit, 5 microseconds for each 1 bit.

The gpr instruction

This draws the right half of a character starting from the current display location, which must not have been changed after the preceedubg gpl instruction.

The IO register is used.

722027

IO - bits 0-17 contain the final 18 bits of the 35 bit character.

Completion pulses will are supported and should be used.

This instruction takes a variable length of time to complete, 2 microseconds for each 0 bit, 5 microseconds for each 1 bit.

After drawing is complete, the x positon will be that of the next character to write, the y position will be reset to the baseline position initially set.

Character matrix

As noted, each 5x7 character is composed of two words. The character dot cells are in this bit order in the words, the first word handled by gpl, the second by gpr:

6	13	3	10	17
5	12	2	9	16
4	11	1	8	15
3	10	0	7	14
2	9	16	6	13
1	8	15	5	12
0	7	14	4	11

Also note again that only 17 bits from the first word are used, bit 17 is the subscript flag.

Spacing and timing

The inter-dot spacing within a character is 2 pixels + size, 2 - 5 pixels.

The bottom of a subscripted character is shifted down 2 dot-spacings below the baseline.

If auto-spacing is used, the spacing between characters is 4 pixels + size, 4 - 7 pixels.

Timing of `gpl` and `gpr` is variable as stated above. According to the DEC manuals, a character containing 16 'on' bits takes approximately 140 microseconds to completely render. This of course does not include the program logic to actually feed characters to the symbol generator, which can add considerable extra time, nor does it include the setup time for setting the drawing origin and format.

Again, you must be sure a `gpl` has completed before issuing a `gpr` or the character will not be properly drawn.

Simple example

This is the basic structure used to draw characters, **am1** syntax, but of course **macro** could be used:

```
lio [6 // auto-space, size 2
glf

top,
lio y-coord
lac x-coord
sdb 200 // starting position, brightness 2
lio first-word-of-char
gpl; ioh
lio second-word-of-char
gpr; ioh

Repeat for each successive character

Other program code

jmp top
```

There are several **am1** include files to define flexo character to word-pair mappings for all flexo characters, as well as subroutines to display one flex character, a line of characters created by the *text* pseudo-instruction, display a digit 0-9, and unpack a *text* string into a buffer of word-pairs and render it.

The unpack routines provide the fastest way to display a sequence of characters, but require more memory space.

